READ ME

CASE STUDY COLLECTION

Name	Chirivella Goutham Sriraj
Roll Number	26001453
Branch	CSD
Section	2

Case Studies

1	Case Study 11: Examination Result and Grace-Mark Validation
2	Case Study 12: Attendance Recovery Computation
3	Case Study 13: ATM Transaction Sequence Simulation
4	Case Study 25: Progressive Water Tariff Analysis
5	Case Study 18: Fuel Consumption and Route Comparison
6	Case Study 39: Multi-Stage Examination Eligibility System
7	Case Study 1: University Scholarship Allocation
8	Case Study 23: Student Timetable Conflict Detection
9	Case Study 8: Manufacturing Cost and Profitability Analysis
10	Case Study 65: Campus Resource Allocation Under Constraints

END OF README

Case Study 1: Examination Result and Grace-Mark Validation

Objective

Create a Python program that accepts marks in multiple subjects, checks subject-wise pass criteria, applies a limited grace-mark policy, prevents marks from exceeding the maximum, and prints the final result and classification.

Rules and Assumptions

• There are 5 subjects, each with a maximum of 100 marks.

• The pass mark is 35 in each subject.

• Grace policy for this example: if exactly one subject is short of 35 by 1 to 5 marks, add only the marks needed to bring that subject to 35. No more than 5 grace marks are awarded in total.

• Grace marks are not awarded if more than one subject is below 35, or if a subject is short by more than 5 marks.

• Marks are validated to be between 0 and 100. Final marks are capped at 100.

• Classification is based on the final aggregate percentage, but a student must pass every subject to receive a pass classification: Distinction = 75% or more; First Class = 60% to below 75%; Second Class = 50% to below 60%; Pass = 35% to below 50%; otherwise Fail.

Algorithm

1. Start.

2. Set the number of subjects, maximum marks, pass mark, and grace-mark limit.

3. Read each subject's marks and validate that they are between 0 and 100.

4. Find the subjects below the pass mark.

5. If exactly one subject is below 35 and it needs at most 5 marks, add the required grace marks.

6. Ensure the final marks do not exceed the maximum marks.

7. Check whether every subject has reached the pass mark.

8. Calculate total marks and aggregate percentage.

9. Assign the final classification using the rules above.

10. Display original marks, final marks, grace marks used, total, percentage, and classification.

11. Stop.

Pseudocode

```
START
    subjects = 5
    pass_mark = 35
    grace_limit = 5
    READ marks for each subject
    VALIDATE every mark is between 0 and 100
    failed_subjects = subjects with marks below pass_mark

    IF number of failed_subjects == 1:
        shortfall = pass_mark - marks of that subject
        IF shortfall <= grace_limit:
            add shortfall as grace marks
            update that subject's final marks
        END IF
    END IF

    IF every final subject mark >= pass_mark:
        calculate total and percentage
        assign classification from percentage
    ELSE:
```

Complete Python Solution

```
NUMBER_OF_SUBJECTS = 5
MAX_MARKS = 100
PASS_MARK = 35
GRACE_LIMIT = 5

subjects = ["English", "Mathematics", "Physics", "Chemistry", "Computer Science"]
marks = []

# Read and validate marks
for subject in subjects:
    while True:
        try:
            mark = int(input("Enter marks in " + subject + " (0-100): "))
            if 0 <= mark <= MAX_MARKS:
                marks.append(mark)
                break
            else:
                print("Invalid marks. Enter a value from 0 to 100.")
        except ValueError:
            print("Please enter a whole number.")

final_marks = marks.copy()
grace_used = [0] * NUMBER_OF_SUBJECTS

# Identify subjects below the pass mark
failed_indices = []
for i in range(NUMBER_OF_SUBJECTS):
    if final_marks[i] < PASS_MARK:
        failed_indices.append(i)

# Apply grace only when exactly one subject is short by at most 5 marks
if len(failed_indices) == 1:
    i = failed_indices[0]
    shortfall = PASS_MARK - final_marks[i]

    if shortfall <= GRACE_LIMIT:
        grace_used[i] = shortfall
        final_marks[i] = min(final_marks[i] + shortfall, MAX_MARKS)

total = sum(final_marks)
maximum_total = NUMBER_OF_SUBJECTS * MAX_MARKS
percentage = (total / maximum_total) * 100

# A student must pass every subject to receive a pass classification
all_subjects_passed = True
for mark in final_marks:
    if mark < PASS_MARK:
        all_subjects_passed = False

if not all_subjects_passed:
    classification = "Fail"
elif percentage >= 75:
    classification = "Distinction"
elif percentage >= 60:
    classification = "First Class"
elif percentage >= 50:
    classification = "Second Class"
elif percentage >= 35:
    classification = "Pass"
else:
    classification = "Fail"

print("\\n----- EXAMINATION RESULT -----")
for i in range(NUMBER_OF_SUBJECTS):
    print(subjects[i] + ": Original =", marks[i],
          "| Grace =", grace_used[i],
          "| Final =", final_marks[i])

print("Total:", total, "/", maximum_total)
print("Percentage: {:.2f}%".format(percentage))
print("Classification:", classification)
```

Sample Run

```
Enter marks in English (0-100): 78
```

```
Enter marks in Mathematics (0-100): 33
Enter marks in Physics (0-100): 69
Enter marks in Chemistry (0-100): 81
Enter marks in Computer Science (0-100): 74

----- EXAMINATION RESULT -----
English: Original = 78 | Grace = 0 | Final = 78
Mathematics: Original = 33 | Grace = 2 | Final = 35
Physics: Original = 69 | Grace = 0 | Final = 69
Chemistry: Original = 81 | Grace = 0 | Final = 81
Computer Science: Original = 74 | Grace = 0 | Final = 74
Total: 337 / 500
Percentage: 67.40%
Classification: First Class
```

Explanation of the Program

• **Lists:** `subjects`, `marks`, `final_marks`, and `grace_used` store related values.

• **Loops:** `for` loops read marks, identify failed subjects, and check whether all subjects passed. A `while` loop repeats input until it is valid.

• **Conditional statements:** `if/elif/else` applies the grace policy and determines the classification.

• **Operators:** Comparison operators validate marks; arithmetic operators calculate totals and percentage; `min()` ensures marks never exceed the maximum.

• **Error handling:** `try/except` handles non-integer input. Invalid marks are rejected rather than included in the result.

Test Cases

Original marks	Grace applied?	Expected result
78, 33, 69, 81, 74	2 marks to Mathematics	337/500, 67.40%, First Class
78, 29, 69, 81, 74	No; shortfall is 6	Fail
78, 33, 29, 81, 74	No; two subjects failed	Fail
100, 100, 100, 100, 100	No	500/500, 100%, Distinction

Conclusion: This program demonstrates input validation, variables, lists, loops, conditional statements, operators, and basic computational thinking. The grace-mark policy and classification thresholds are example rules; replace them with the official rules if your institution specifies different ones.

Case Study 12: Attendance Recovery Computation

Objective

Write a Python program to calculate a student's current attendance percentage and find the minimum number of consecutive future classes the student must attend to reach a required attendance threshold. The program also compares different future attendance scenarios.

Rules and Assumptions

• Classes held and classes attended are entered as whole numbers.

• Classes attended cannot be greater than classes held; both values must be non-negative.

• The eligibility threshold is entered by the user as a percentage, for example 75%.

• For recovery, assume the student attends every next class. If the current attendance is already at or above the threshold, zero extra classes are required.

• The minimum number of classes is calculated using a loop, so the result is an exact whole number and does not rely on rounding.

• Future scenarios compare attending all upcoming classes, attending half of them (rounded down), and missing all upcoming classes.

Algorithm

1. Start.

2. Input classes held, classes attended, and the required attendance percentage.

3. Validate the inputs.

4. Calculate current attendance = (classes attended / classes held) × 100. If no classes have been held, report that the percentage is not yet defined.

5. If current attendance meets the threshold, set required recovery classes to zero.

6. Otherwise, repeatedly add one attended class and one class held until the threshold is reached.

7. Calculate future attendance under the selected scenarios.

8. Display current attendance, recovery requirement, and scenario results.

9. Stop.

Pseudocode

```
START
    INPUT classes_held, classes_attended, threshold
    VALIDATE inputs

    IF classes_held == 0:
        current_percentage = undefined
    ELSE:
        current_percentage = classes_attended / classes_held * 100

    IF current_percentage >= threshold:
        required_classes = 0
    ELSE:
        future_held = classes_held
        future_attended = classes_attended
        required_classes = 0

        WHILE future_attended / future_held * 100 < threshold:
            future_held = future_held + 1
            future_attended = future_attended + 1
            required_classes = required_classes + 1

    CALCULATE scenario percentages
    DISPLAY results
END
```

Flowchart (text form)

Complete Python Solution

```
def attendance_percentage(attended, held):
    if held == 0:
        return None
    return (attended / held) * 100


def future_percentage(attended, held, future_classes, attend_count):
    # attend_count is the number of future classes the student attends
    total_attended = attended + attend_count
    total_held = held + future_classes

    if total_held == 0:
        return None

    return (total_attended / total_held) * 100


while True:
    try:
        held = int(input("Enter total classes held: "))
        attended = int(input("Enter classes attended: "))
        threshold = float(input("Enter eligibility threshold (%): "))

        if held < 0 or attended < 0:
            print("Class counts cannot be negative.")
        elif attended > held:
            print("Classes attended cannot exceed classes held.")
        elif not 0 <= threshold <= 100:
            print("Threshold must be between 0 and 100.")
        else:
            break
    except ValueError:
        print("Enter whole numbers for class counts and a number for threshold.")

current = attendance_percentage(attended, held)

print("\\n----- ATTENDANCE REPORT -----")
if current is None:
    print("Current attendance: Not defined (no classes held yet)")
else:
    print("Current attendance: {:.2f}%".format(current))

# Find minimum consecutive classes to attend to reach the threshold
required = 0
future_held = held
future_attended = attended

if current is None:
    # With no classes held, the first attended class starts attendance at 100%.
    if threshold > 0:
        required = 1
        future_held = 1
        future_attended = 1
else:
    while (future_attended / future_held * 100) < threshold:
        future_held += 1
        future_attended += 1
        required += 1

print("Consecutive classes to attend for eligibility:", required)

if future_held > 0:
    recovered = (future_attended / future_held) * 100
    print("Attendance after recovery: {:.2f}%".format(recovered))

# Compare scenarios for the next 10 classes
future_classes = 10
scenarios = [
    ("Attend all 10 classes", 10),
    ("Attend 5 of the next 10 classes", 5),
    ("Miss all 10 classes", 0)
]

print("\\n----- FUTURE SCENARIOS (NEXT 10 CLASSES) -----")
for name, attend_count in scenarios:
    result = future_percentage(attended, held, future_classes, attend_count)
    if result is None:
        print(name + ": Not defined")
```

```
    else:
        status = "Eligible" if result >= threshold else "Not eligible"
        print("{}: {:.2f}% - {}".format(name, result, status))
```

Sample Run

```
Enter total classes held: 40
Enter classes attended: 26
Enter eligibility threshold (%): 75

----- ATTENDANCE REPORT -----
Current attendance: 65.00%
Consecutive classes to attend for eligibility: 16
Attendance after recovery: 75.00%

----- FUTURE SCENARIOS (NEXT 10 CLASSES) -----
Attend all 10 classes: 72.00% - Not eligible
Attend 5 of the next 10 classes: 62.00% - Not eligible
Miss all 10 classes: 52.00% - Not eligible
```

Line-by-Line Explanation

• **Functions:** `attendance_percentage()` calculates the current percentage, while `future_percentage()` calculates attendance after a future scenario.

• **Input validation:** The `while` loop repeats until the class counts and threshold are valid.

• **Operators:** Division and multiplication calculate percentages; comparisons check the eligibility threshold.

• **Recovery loop:** The program adds one attended class and one class held at a time until the threshold is reached. This guarantees the minimum consecutive number of classes.

• **Tuples and unpacking:** Each scenario is stored as a tuple containing a description and the number of classes attended. The loop unpacks each tuple into `name` and `attend_count`.

• **Conditional statements:** The program prints whether each future scenario meets the threshold.

Test Cases

Held	Attended	Threshold	Minimum consecutive classes
40	26	75%	16
40	30	75%	0 (already eligible)
20	10	75%	50
0	0	75%	1 (first class attended)

Conclusion: This case study uses variables, data types, arithmetic and comparison operators, loops, conditional statements, functions, tuples, and tuple unpacking to calculate attendance recovery. The eligibility threshold is user-defined; use the rule set by your institution.

Case Study 13: ATM Transaction Sequence Simulation

Objective

Develop a Python program that processes a sequence of ATM deposits and withdrawals, maintains the account's simulated balance, enforces a withdrawal limit, rejects invalid transactions, and prints a transaction summary.

Rules and Assumptions

• The starting balance is entered by the user and must be non-negative.

• The maximum amount for a single withdrawal is ■10,000.

• Deposits must be greater than zero.

• Withdrawals must be greater than zero, must not exceed ■10,000, and must not exceed the available balance.

• Invalid transaction types or invalid amounts are rejected and do not change the balance.

• The simulation ends when the user enters Q to quit, or after the specified number of transaction attempts.

• This is an educational simulation; it does not connect to a real bank account.

Algorithm

1. Start.

2. Input the starting balance and the maximum number of transaction attempts.

3. Set deposit, withdrawal, and rejected-transaction counters to zero.

4. Repeatedly ask for a transaction type: D for deposit, W for withdrawal, or Q to quit.

5. For D or W, input the amount and validate it.

6. For a valid deposit, add the amount to the balance.

7. For a valid withdrawal, check the per-transaction limit and available balance before subtracting.

8. Record each attempt in a transaction list, including rejected attempts.

9. Display the final balance and a summary of successful and rejected transactions.

10. Stop.

Pseudocode

```
START
    INPUT starting_balance
    balance = starting_balance
    deposits = 0
    withdrawals = 0
    rejected = 0
    transactions = empty list

    REPEAT for each transaction attempt:
        INPUT transaction_type

        IF type is Q:
            EXIT loop
        ELSE IF type is D or W:
            INPUT amount
            IF amount <= 0:
                reject transaction
            ELSE IF type is D:
                balance = balance + amount
                deposits = deposits + 1
                record success
            ELSE IF amount > withdrawal_limit:
                reject transaction
            ELSE IF amount > balance:
                reject transaction
            ELSE:
                balance = balance - amount
                withdrawals = withdrawals + 1
                record success
        ELSE:
            reject invalid transaction type
```

Complete Python Solution

```
WITHDRAWAL_LIMIT = 10000

while True:
    try:
        balance = float(input("Enter starting balance (■): "))
        if balance < 0:
            print("Starting balance cannot be negative.")
        else:
            break
    except ValueError:
        print("Please enter a valid number.")

while True:
    try:
        max_attempts = int(input("Maximum transaction attempts: "))
        if max_attempts <= 0:
            print("Enter at least 1 attempt.")
        else:
            break
    except ValueError:
        print("Please enter a whole number.")

starting_balance = balance
deposit_count = 0
withdrawal_count = 0
rejected_count = 0
total_deposited = 0.0
total_withdrawn = 0.0
transactions = []

for attempt in range(1, max_attempts + 1):
    transaction_type = input(
        "Transaction " + str(attempt) +
        " - Enter D (deposit), W (withdraw), or Q (quit): "
    ).strip().upper()

    if transaction_type == "Q":
        print("Simulation ended by user.")
        break

    if transaction_type not in ("D", "W"):
        print("Rejected: invalid transaction type.")
        rejected_count += 1
        transactions.append((attempt, transaction_type, 0, "Rejected: invalid type", balance))
        continue

    try:
        amount = float(input("Enter amount (■): "))
    except ValueError:
        print("Rejected: amount must be numeric.")
        rejected_count += 1
        transactions.append((attempt, transaction_type, 0, "Rejected: invalid amount", balance))
        continue

    if amount <= 0:
        status = "Rejected: amount must be positive"
        rejected_count += 1

    elif transaction_type == "D":
        balance += amount
        deposit_count += 1
        total_deposited += amount
        status = "Deposit successful"

    elif amount > WITHDRAWAL_LIMIT:
        status = "Rejected: exceeds withdrawal limit"
        rejected_count += 1

    elif amount > balance:
        status = "Rejected: insufficient balance"
        rejected_count += 1

    else:
        balance -= amount
        withdrawal_count += 1
        total_withdrawn += amount
        status = "Withdrawal successful"

    transactions.append((attempt, transaction_type, amount, status, balance))
    print(status + ". Current balance: ■{:.2f}".format(balance))

print("\\n========== TRANSACTION SUMMARY ==========")
```

```
print("Starting balance: ■{:.2f}".format(starting_balance))
print("Final balance: ■{:.2f}".format(balance))
print("Successful deposits:", deposit_count)
print("Successful withdrawals:", withdrawal_count)
print("Rejected transactions:", rejected_count)
print("Total deposited: ■{:.2f}".format(total_deposited))
print("Total withdrawn: ■{:.2f}".format(total_withdrawn))

print("\\nTransaction history:")
for number, kind, amount, status, remaining in transactions:
    print("#{} | Type: {} | Amount: ■{:.2f} | {} | Balance: ■{:.2f}".format(
        number, kind, amount, status, remaining
    ))
```

Sample Run

```
Enter starting balance (■): 5000
Maximum transaction attempts: 4
Transaction 1 - Enter D (deposit), W (withdraw), or Q (quit): D
Enter amount (■): 2000
Deposit successful. Current balance: ■7000.00
Transaction 2 - Enter D (deposit), W (withdraw), or Q (quit): W
Enter amount (■): 1500
Withdrawal successful. Current balance: ■5500.00
Transaction 3 - Enter D (deposit), W (withdraw), or Q (quit): W
Enter amount (■): 12000
Rejected: exceeds withdrawal limit. Current balance: ■5500.00
Transaction 4 - Enter D (deposit), W (withdraw), or Q (quit): W
Enter amount (■): 7000
Rejected: insufficient balance. Current balance: ■5500.00

========== TRANSACTION SUMMARY ==========
Starting balance: ■5000.00
Final balance: ■5500.00
Successful deposits: 1
Successful withdrawals: 1
Rejected transactions: 2
Total deposited: ■2000.00
Total withdrawn: ■1500.00
```

Explanation of the Program

• **Variables and data types:** Numeric variables store the balance and totals; strings store transaction types and statuses.

• **Loops:** `while` loops validate the starting balance and attempt count. A `for` loop processes each transaction.

• **Conditional statements:** `if/elif/else` checks transaction type, positive amounts, withdrawal limit, and available balance.

• **Lists:** `transactions` stores the history of all attempts.

• **Tuples and unpacking:** Each history record is a tuple. The final loop unpacks its five values into variables for display.

• **Operators:** Arithmetic operators update the balance and totals; comparison operators enforce the rules.

• **Validation:** Invalid input or rejected transactions never change the account balance.

Test Cases

Transaction	Expected behavior
Deposit ■2,000	Balance increases by ■2,000
Withdraw ■1,500 with enough balance	Balance decreases by ■1,500
Withdraw ■12,000	Rejected: above ■10,000 limit
Withdraw more than available balance	Rejected: insufficient balance
Deposit ■0 or a negative amount	Rejected: amount must be positive
Enter unknown transaction type	Rejected: invalid type; balance unchanged

Conclusion: This educational simulation uses variables, strings, arithmetic and comparison operators, conditional statements, loops, lists, tuples, and unpacking to process ATM transactions safely. The withdrawal limit is an example rule and can be changed as required.

Case Study 25

Progressive Water Tariff Analysis

Complete Python Solution Using Only the Assigned Syllabus

1. Problem Definition

A water utility needs to calculate monthly bills for multiple households. The bill depends on progressive consumption slabs, a fixed service fee, and a penalty for excessive usage. The program must compare household consumption, identify households that exceed a chosen usage threshold, and report the lowest and highest bills. Use lists, tuples, loops, conditionals, operators, strings, and tuple unpacking only; no external libraries are required for the solution.

2. Assumptions and Tariff Rules

Item	Rule used in this example
Slab 1	First 10 kL: ■5 per kL
Slab 2	Next 10 kL (more than 10 up to 20): ■8 per kL
Slab 3	Above 20 kL: ■12 per kL
Fixed fee	■50 per household per month
Excessive-use threshold	More than 25 kL in a month
Penalty	■100 if consumption is above 25 kL

Note: These rates are illustrative classroom values, not an official local water tariff. Consumption is measured in kilolitres (kL). Each slab is charged only for the units that fall inside that slab.

3. Computational Thinking

Decomposition: collect household names and usage, calculate slab charges, add fees/penalties, then compare results.
Pattern recognition: every household follows the same billing rules.
Abstraction: use fixed rates and a threshold instead of modeling a real utility's entire billing system.
Algorithm design: process each household in a loop and store its result as a tuple.

4. Algorithm

1. Start.
2. Store household names and monthly water consumption in lists.
3. Set slab rates, fixed fee, excessive-use threshold, and penalty.
4. For each household, calculate charges for the first 10 kL, next 10 kL, and usage above 20 kL.
5. Add the fixed fee; if usage exceeds the threshold, add the penalty.
6. Store (name, consumption, slab charge, penalty, total bill) in a tuple.
7. Display each bill and mark excessive usage.
8. Compare consumption and bills; identify minimum and maximum bill.
9. Stop.

5. Pseudocode

```
START
  names = household names
  usage = monthly consumption values
  results = empty list

  FOR each household index i
      units = usage[i]
      IF units <= 10
          slab_charge = units * 5
      ELSE IF units <= 20
          slab_charge = 10 * 5 + (units - 10) * 8
      ELSE
          slab_charge = 10 * 5 + 10 * 8 + (units - 20) * 12
      END IF

      penalty = 0
      IF units > 25
          penalty = 100
      END IF

      total = slab_charge + 50 + penalty
      APPEND (names[i], units, slab_charge, penalty, total) to results
  END FOR

  DISPLAY each result
  FIND result with minimum total bill
  FIND result with maximum total bill
  DISPLAY households above 25 kL
END
```

```
RATE_3 = 12
FIXED_FEE = 50
EXCESS_THRESHOLD = 25
PENALTY = 100

results = []

for i in range(len(names)):
    units = usage[i]

    # Calculate progressive slab charges.
    if units <= 10:
        slab_charge = units * RATE_1
    elif units <= 20:
        slab_charge = (10 * RATE_1) + ((units - 10) * RATE_2)
    else:
        slab_charge = (10 * RATE_1) + (10 * RATE_2) \\
                      + ((units - 20) * RATE_3)

    # Apply excessive-use penalty.
    if units > EXCESS_THRESHOLD:
        penalty = PENALTY
        status = "EXCESSIVE"
    else:
        penalty = 0
        status = "Normal"

    total_bill = slab_charge + FIXED_FEE + penalty

    # Tuple stores one household's complete result.
    record = (names[i], units, slab_charge, penalty, total_bill, status)
    results.append(record)

print("PROGRESSIVE WATER TARIFF REPORT")
print("-" * 78)
print("{:<10} {:>8} {:>12} {:>10} {:>12} {:>12}".format(
    "Household", "Usage", "Slab charge", "Penalty", "Total bill", "Status"))
print("-" * 78)

total_usage = 0
total_revenue = 0
excessive_count = 0

for record in results:
    name, units, slab_charge, penalty, total_bill, status = record  # unpacking
    print("{:<10} {:>8} {:>12.2f} {:>10.2f} {:>12.2f} {:>12}".format(
        name, units, slab_charge, penalty, total_bill, status))
    total_usage = total_usage + units
    total_revenue = total_revenue + total_bill

    if status == "EXCESSIVE":
        excessive_count = excessive_count + 1

# Find lowest and highest bills using a loop.
lowest = results[0]
highest = results[0]

for record in results:
    if record[4] < lowest[4]:
        lowest = record
    if record[4] > highest[4]:
        highest = record

print("-" * 78)
print("Households:", len(names))
print("Total water consumed:", total_usage, "kL")
print("Average consumption:", round(total_usage / len(names), 2), "kL")
print("Total revenue:", "Rs.", round(total_revenue, 2))
print("Households with excessive use:", excessive_count)
print("Lowest bill:", lowest[0], "- Rs.", round(lowest[4], 2))
print("Highest bill:", highest[0], "- Rs.", round(highest[4], 2))

print("\nEXCESSIVE-USAGE HOUSEHOLDS")
found = False
for record in results:
    if record[5] == "EXCESSIVE":
        print(record[0], "used", record[1], "kL")
        found = True
if not found:
    print("No household exceeded the threshold.")
```

8. Sample Input and Expected Output

The sample data are built into the program: Asha 8 kL, Bharat 15 kL, Charan 22 kL, Divya 28 kL, Eshan 12 kL, and Farah 35 kL.

```
PROGRESSIVE WATER TARIFF REPORT
--------------------------------------------------------------------------------
Household     Usage  Slab charge    Penalty   Total bill       Status
--------------------------------------------------------------------------------
Asha              8        40.00       0.00        90.00       Normal
Bharat           15        90.00       0.00       140.00       Normal
Charan           22       154.00       0.00       204.00       Normal
Divya            28       226.00     100.00       376.00       EXCESSIVE
Eshan            12        66.00       0.00       116.00       Normal
Farah            35       310.00     100.00       460.00       EXCESSIVE
--------------------------------------------------------------------------------
Households: 6
Total water consumed: 120 kL
Average consumption: 20.0 kL
Total revenue: Rs. 1386.0
Households with excessive use: 2
Lowest bill: Asha - Rs. 90.0
Highest bill: Farah - Rs. 460.0

EXCESSIVE-USAGE HOUSEHOLDS
Divya used 28 kL
Farah used 35 kL
```

9. Worked Calculation

Example: Divya uses 28 kL.
First 10 kL: 10 × ■5 = ■50
Next 10 kL: 10 × ■8 = ■80
Remaining 8 kL: 8 × ■12 = ■96
Slab charge = ■50 + ■80 + ■96 = ■226
Fixed fee = ■50; usage is above 25 kL, so penalty = ■100.
Total bill = ■226 + ■50 + ■100 = ■376.

10. Test Cases

Usage	Expected slab charge	Penalty	Total bill	Reason
0 kL	■0	■0	■50	No consumption; fixed fee applies
10 kL	■50	■0	■100	Upper edge of slab 1
20 kL	■130	■0	■180	Upper edge of slab 2
25 kL	■190	■0	■240	Threshold itself is not penalized
26 kL	■202	■100	■352	Above threshold; penalty applies

11. Syllabus Concepts Used

• Computational thinking: decomposition, abstraction, repeated processing, and comparison.

• Algorithm and pseudocode: clear step-by-step billing procedure.

• Variables and data types: strings, integers, and floating-point bill values.

• Operators: arithmetic (+, -, *, /) and comparisons (<=, >, <).

• Conditional statements: if, elif, and else for slabs and penalties.

• Loops: for loops to calculate and summarize every household.

• Strings: household names, status messages, and formatted report headings.

• Lists: names, usage values, and a list of billing records.

• Tuples and tuple operations: each result is stored as a tuple and indexed for comparisons.

• Unpacking: one tuple is unpacked into named variables before printing.

• Pattern generation: repeated hyphens create report separators.

12. Conclusion

The program calculates progressive water bills, applies a fixed fee and excessive-use penalty, compares household consumption, and identifies the lowest and highest bills. The solution uses only the listed introductory syllabus concepts and standard Python built-ins.

FUEL CONSUMPTION AND ROUTE COMPARISON

Case Study with Complete Python Solution

Course area: Introductory Python Programming

Topics: Computational thinking, algorithms, pseudocode, variables, data types, operators, conditionals, loops, strings, lists, nested lists, tuples, tuple operations, unpacking, and pattern generation.

Objective

Compare vehicle options for a planned journey by estimating fuel needed, reserve fuel, total fuel to carry, and fuel cost. The program checks whether each vehicle has enough usable fuel capacity and identifies the lowest-cost feasible option.

Assumptions

Distance is entered in kilometres, mileage in kilometres per litre (km/L), tank capacity and reserve in litres, and fuel price in currency units per litre. The required reserve is an additional amount kept after completing the journey. A vehicle is feasible if its tank capacity can hold the journey fuel requirement plus the reserve. The model does not include tolls, traffic, or refuelling stops.

1. Problem Definition and Formulae

Measure	Formula
Journey fuel	Distance ÷ mileage
Reserve fuel	Journey fuel × reserve percentage ÷ 100
Fuel required including reserve	Journey fuel + reserve fuel
Fuel cost	Journey fuel × fuel price per litre
Feasibility	Fuel required including reserve ≤ tank capacity

2. Algorithm

```
1. Read the journey distance and reserve percentage.
2. Read the number of vehicles to compare.
3. For each vehicle, read name, mileage, tank capacity, and fuel price.
4. Calculate journey fuel and reserve fuel.
5. Calculate fuel needed including reserve and estimated journey cost.
6. Mark a vehicle feasible if its capacity covers the full fuel requirement.
7. Store each vehicle's results in a tuple inside a list.
8. Find the feasible vehicle with the lowest journey fuel cost.
9. Display all results and the recommended feasible option.
```

3. Pseudocode

```
START
  INPUT distance, reserve_percent, vehicle_count
  results = empty list

  FOR each vehicle:
      INPUT name, mileage, tank_capacity, fuel_price
      journey_fuel = distance / mileage
      reserve_fuel = journey_fuel * reserve_percent / 100
      total_required = journey_fuel + reserve_fuel
      journey_cost = journey_fuel * fuel_price

      IF total_required <= tank_capacity:
          feasible = True
      ELSE:
          feasible = False

      APPEND vehicle details and calculations to results

  best_vehicle = none
  FOR each result:
      IF feasible AND (best_vehicle is none OR cost < best cost):
          best_vehicle = current result

  DISPLAY all vehicle calculations
  IF best_vehicle exists:
      DISPLAY lowest-cost feasible option
  ELSE:
      DISPLAY no feasible vehicle
END
```

4. Complete Python Program

```
# Fuel Consumption and Route Comparison
# Introductory Python only: variables, operators, conditionals,
# loops, strings, lists, tuples, and tuple unpacking.

def read_number(prompt, minimum=0, allow_zero=True):
    while True:
        try:
            value = float(input(prompt))
            if value < minimum or (not allow_zero and value == 0):
                print("Enter a valid value.")
            else:
                return value
        except ValueError:
            print("Invalid input. Enter a number.")


def main():
    distance = read_number("Journey distance in kilometres: ", 0, False)
    reserve_percent = read_number(
        "Required reserve percentage (e.g. 15 for 15%): ", 0
    )
    while reserve_percent > 100:
        print("Reserve percentage should be between 0 and 100.")
        reserve_percent = read_number(
            "Required reserve percentage (0-100): ", 0
        )

    vehicle_count = int(read_number("Number of vehicles to compare: ", 1, False))
    results = []

    for i in range(vehicle_count):
        print("\\nVEHICLE", i + 1)
        name = input("Vehicle name: ").strip()
        if name == "":
            name = "Vehicle " + str(i + 1)

        mileage = read_number("Mileage in km per litre: ", 0, False)
        tank_capacity = read_number("Usable tank capacity in litres: ", 0)
        fuel_price = read_number("Fuel price per litre: ", 0)

        journey_fuel = distance / mileage
        reserve_fuel = journey_fuel * reserve_percent / 100
        total_required = journey_fuel + reserve_fuel
        journey_cost = journey_fuel * fuel_price

        if total_required <= tank_capacity:
            feasible = True
            status = "Feasible"
        else:
            feasible = False
            status = "Not feasible"

        # Store each result as a tuple.
        results.append((
            name, mileage, tank_capacity, fuel_price,
            journey_fuel, reserve_fuel, total_required,
            journey_cost, feasible, status
        ))

    print("\\nFUEL AND COST COMPARISON")
    print("=" * 100)
    print(
        "Vehicle".ljust(18), "Mileage".rjust(9), "Tank(L)".rjust(9),
        "Trip fuel".rjust(11), "Reserve".rjust(10),
        "Total need".rjust(11), "Cost".rjust(12), "Status"
    )
    print("-" * 100)

    best = None
    for result in results:
        (name, mileage, tank_capacity, fuel_price, journey_fuel,
         reserve_fuel, total_required, journey_cost, feasible, status) = result

        print(
            name[:18].ljust(18),
            f"{mileage:9.2f}",
            f"{tank_capacity:9.2f}",
            f"{journey_fuel:11.2f}",
            f"{reserve_fuel:10.2f}",
            f"{total_required:11.2f}",
            f"{journey_cost:12.2f}",
            status
        )

        # Select the lowest-cost vehicle among feasible options only.
```

```
            if feasible:
                if best is None or journey_cost < best[7]:
                    best = result

    print("=" * 100)
    if best is None:
        print("No vehicle is feasible with the current tank and reserve requirements.")
    else:
        (name, mileage, tank_capacity, fuel_price, journey_fuel,
         reserve_fuel, total_required, journey_cost, feasible, status) = best
        print("LOWEST-COST FEASIBLE OPTION:", name)
        print("Estimated journey fuel:", f"{journey_fuel:.2f}", "litres")
        print("Reserve fuel:", f"{reserve_fuel:.2f}", "litres")
        print("Fuel required including reserve:", f"{total_required:.2f}", "litres")
        print("Estimated journey fuel cost:", f"{journey_cost:.2f}")

    print("\\nNote: cost is based on journey fuel consumed, not reserve fuel that remains unused.")


if __name__ == "__main__":
    main()
```

The program assumes a single tank can cover the journey and the specified reserve without refuelling. It compares fuel costs only; if two feasible vehicles have the same cost, the first entered remains the recommendation.

Test case	Expected behavior
Mileage is zero	Rejected; mileage must be positive.
Reserve is 0%	Reserve fuel is zero.
No feasible vehicles	Program reports no feasible option.
Several feasible vehicles	Lowest estimated journey fuel cost is selected.
Two vehicles have equal costs	First feasible vehicle entered is retained.

8. Syllabus Concepts Demonstrated

• Computational thinking: breaking the task into input, calculation, feasibility checking, and comparison.
• Variables and data types: strings for vehicle names, numeric values for distance, mileage, capacity, and prices; booleans for feasibility.
• Operators: arithmetic and comparison operators.
• Conditional statements: identifying feasible vehicles and selecting the minimum-cost option.
• Loops: repeated vehicle input and comparison.
• Strings and pattern generation: formatted labels and separator lines using the * operator.
• Lists and tuples: storing multiple vehicle results as tuples in a list.
• Tuple operations and unpacking: reading each stored result into named variables.

Conclusion

This solution compares multiple vehicles using distance, mileage, fuel price, tank capacity, and a reserve requirement. It selects the lowest-cost option only from vehicles that can complete the journey while preserving the requested reserve.

Case Study 39

Multi-Stage Examination Eligibility System

Complete Python Solution Using Only the Assigned Syllabus

1. Problem Definition

A college must decide whether each student is eligible to sit for the final examination. Eligibility depends on four independent requirements: minimum attendance, completion of required laboratory work, minimum internal-assessment marks, and fee clearance. The program must evaluate all requirements for each student and report every unmet condition, rather than stopping after the first failure. It should also summarize how many students are eligible and list the reasons for ineligibility.

2. Eligibility Rules and Assumptions

Requirement	Rule used in this example
Attendance	At least 75%
Laboratory completion	All required laboratory work must be completed
Internal assessment	At least 40 marks out of 100
Fee clearance	All required fees must be cleared

These thresholds are illustrative. A real institution should replace them with its official examination regulations. Laboratory completion and fee clearance are represented as True/False values.

3. Computational Thinking

Decomposition: check each requirement separately, combine the results, and create a summary.
Pattern recognition: the same four checks are repeated for every student.
Abstraction: represent laboratory completion and fee clearance as Boolean values and use fixed thresholds for marks and attendance.
Algorithm design: collect every failed requirement in a list and determine eligibility only after all checks are complete.

4. Algorithm

1. Start and define the minimum attendance and internal-assessment marks.
2. Store student records in a list of tuples: name, attendance, lab completion, internal marks, fee clearance.
3. For each student, create an empty list of unmet requirements.
4. Check attendance; add a reason if attendance is below 75%.
5. Check laboratory completion; add a reason if work is incomplete.
6. Check internal marks; add a reason if marks are below 40.
7. Check fee clearance; add a reason if fees are not cleared.
8. If no reasons were added, mark the student eligible; otherwise mark the student not eligible.
9. Display every student's result and all unmet requirements.
10. Count eligible and ineligible students, then stop.

5. Pseudocode

```
START
  students = list of student tuples
  eligible_count = 0
  ineligible_count = 0

  FOR each student_record in students
      UNPACK name, attendance, lab_done, marks, fees_cleared
      unmet = empty list

      IF attendance < 75
          APPEND "Attendance below 75%" to unmet
      END IF
      IF lab_done is False
```

```
            APPEND "Laboratory work incomplete" to unmet
        END IF
        IF marks < 40
            APPEND "Internal marks below 40" to unmet
        END IF
        IF fees_cleared is False
            APPEND "Fees not cleared" to unmet
        END IF

        IF length(unmet) == 0
            status = "ELIGIBLE"
            eligible_count = eligible_count + 1
        ELSE
            status = "NOT ELIGIBLE"
            ineligible_count = ineligible_count + 1
        END IF

        DISPLAY name, status, all items in unmet
    END FOR

    DISPLAY eligible_count and ineligible_count
END
```

```
              +-----------+
```

7. Complete Python Program

```
# Case Study 39: Multi-Stage Examination Eligibility System
# Only basic Python syllabus concepts are used.

MIN_ATTENDANCE = 75
MIN_INTERNAL_MARKS = 40

# Record format:
# (student name, attendance percentage, lab completed,
#  internal marks out of 100, fees cleared)
students = [
    ("Ananya", 88, True, 72, True),
    ("Bala", 68, True, 65, True),
    ("Chitra", 82, False, 55, True),
    ("Dev", 76, True, 35, False),
    ("Esha", 74, False, 32, False),
    ("Farhan", 91, True, 40, True),
]

results = []
eligible_count = 0
ineligible_count = 0

print("FINAL EXAMINATION ELIGIBILITY REPORT")
print("-" * 76)

for student in students:
    # Tuple unpacking
    name, attendance, lab_done, marks, fees_cleared = student
    unmet = []

    # Evaluate every condition independently.
    if attendance < MIN_ATTENDANCE:
        unmet.append("Attendance below 75%")

    if not lab_done:
        unmet.append("Laboratory work incomplete")

    if marks < MIN_INTERNAL_MARKS:
        unmet.append("Internal marks below 40")

    if not fees_cleared:
        unmet.append("Fees not cleared")

    if len(unmet) == 0:
        status = "ELIGIBLE"
        eligible_count = eligible_count + 1
    else:
        status = "NOT ELIGIBLE"
        ineligible_count = ineligible_count + 1

    # Store a tuple containing the decision and all reasons.
    result = (name, status, attendance, marks, unmet)
    results.append(result)

    print("\nStudent:", name)
    print("Attendance:", str(attendance) + "%")
    print("Internal marks:", marks, "/ 100")
    print("Laboratory completed:", lab_done)
    print("Fees cleared:", fees_cleared)
    print("Decision:", status)

    if len(unmet) == 0:
        print("Unmet requirements: None")
    else:
        print("Unmet requirements:")
        for reason in unmet:
            print("-", reason)

print("\n" + "-" * 76)
print("SUMMARY")
print("Total students:", len(students))
print("Eligible:", eligible_count)
print("Not eligible:", ineligible_count)

print("\nSTUDENTS NEEDING ACTION")
found = False
for result in results:
    name, status, attendance, marks, unmet = result
    if status == "NOT ELIGIBLE":
        found = True
        print(name + ":")
        for reason in unmet:
```

```
            print("  -", reason)

if not found:
    print("All students meet every requirement.")
```

8. Sample Data and Expected Results

The program uses six example students. The table below summarizes the decision and every unmet requirement. A student is eligible only when all four requirements are satisfied.

Student	Attendance	Lab	Marks	Fees	Decision / unmet requirements
Ananya	88%	Done	72	Cleared	Eligible
Bala	68%	Done	65	Cleared	Not eligible: attendance below 75%
Chitra	82%	Incomplete	55	Cleared	Not eligible: laboratory work incomplete
Dev	76%	Done	35	Not cleared	Not eligible: marks below 40; fees not cleared
Esha	74%	Incomplete	32	Not cleared	Not eligible: attendance, lab, marks, and fees
Farhan	91%	Done	40	Cleared	Eligible

9. Example Output Summary

```
SUMMARY
Total students: 6
Eligible: 2
Not eligible: 4

STUDENTS NEEDING ACTION
Bala:
  - Attendance below 75%
Chitra:
  - Laboratory work incomplete
Dev:
  - Internal marks below 40
  - Fees not cleared
Esha:
  - Attendance below 75%
  - Laboratory work incomplete
  - Internal marks below 40
  - Fees not cleared
```

10. Test Cases

Case	Input condition	Expected decision
1	75% attendance, lab done, 40 marks, fees cleared	Eligible (threshold values pass)
2	74% attendance; all other requirements pass	Not eligible: attendance
3	Attendance passes; lab incomplete	Not eligible: laboratory
4	Attendance/lab/fees pass; 39 marks	Not eligible: internal marks
5	Only fee clearance fails	Not eligible: fees
6	All four conditions fail	Not eligible with four reasons listed

11. Syllabus Concepts Used

• Computational thinking: break eligibility into four independent checks.

• Algorithms and pseudocode: step-by-step decision process.

• Flowchart: visual representation of the checks and final decision.

• Variables and data types: names (strings), attendance/marks (numbers), flags (Booleans).

• Operators: comparisons (<, ==), arithmetic for counters, and Boolean not.

• Conditional statements: independent if statements ensure all unmet conditions are reported.

• Loops: process students and their unmet requirements.

• Strings: decision labels and explanatory messages.

• Lists: student records, results, and unmet requirements.

• Tuples and tuple operations: fixed-format student and result records; indexing/unpacking.

• Unpacking: split each student tuple into named variables.

• Pattern generation: repeated hyphens form report separators.

12. Conclusion

The program makes an examination decision only after checking attendance, laboratory completion, internal assessment, and fee clearance. Because each requirement is tested independently, the report lists every unmet condition for each student. The solution uses only basic Python concepts from the assigned syllabus and can be adapted to official institutional thresholds.

UNIVERSITY SCHOLARSHIP ALLOCATION

Case Study with Complete Python Solution

Course area: Introductory Python Programming

Topics used: Computational thinking, algorithms, pseudocode, variables, data types, operators, conditionals, loops, lists, nested lists, sorting, and tuple unpacking.

Objective

Create a simple scholarship allocation program that checks applicants against academic, attendance, and family-income requirements. Eligible applicants are ranked using aggregate marks, then ties are resolved by subject score, attendance, lower family income, and finally applicant ID.

Important note

The thresholds and tie-breaking rules in this document are illustrative. A real university should replace them with its officially approved policy and verify the program with test cases before use.

1. Problem Definition

Each applicant provides an ID, name, marks in three subjects, attendance percentage, and annual family income. The program calculates the aggregate percentage from the three subject marks, checks every eligibility condition, and ranks only eligible applicants.

Illustrative eligibility rules

Criterion	Rule
Aggregate marks	At least 75% average across the three subjects
Subject minimum	At least 60 marks in each subject (each subject is out of 100)
Attendance	At least 85%
Annual family income	At most ■300,000
Scholarship limit	Award scholarships to the first N eligible applicants; N is entered by the user

Tie-breaking rules (in order)

1. Higher aggregate percentage ranks first.
2. If aggregate percentages are equal, higher lowest-subject mark ranks first.
3. If still tied, higher attendance ranks first.
4. If still tied, lower annual family income ranks first.
5. If still tied, applicant ID in alphabetical/numeric text order ranks first.

2. Algorithm

```
1. Read the number of applicants and the number of scholarships.
2. For each applicant, read ID, name, three subject marks, attendance, and income.
3. Calculate aggregate percentage = (mark1 + mark2 + mark3) / 3.
4. Check aggregate, every subject minimum, attendance, and income.
5. Store the applicant and eligibility result in a list.
6. Put eligible applicants into a list of tuples containing ranking fields.
7. Sort eligible applicants by the tie-breaking rules.
8. Display all applicants with eligibility status, then display ranked winners.
```

3. Pseudocode

```
START
  INPUT applicant_count, scholarship_count
  applicants = empty list

  REPEAT applicant_count times:
      INPUT applicant_id, name
      INPUT mark1, mark2, mark3, attendance, income
      aggregate = (mark1 + mark2 + mark3) / 3
      lowest_subject = minimum(mark1, mark2, mark3)

      eligible = (aggregate >= 75) AND
                 (mark1 >= 60 AND mark2 >= 60 AND mark3 >= 60) AND
                 (attendance >= 85) AND (income <= 300000)

      APPEND applicant details and eligible to applicants

  eligible_list = empty list
  FOR each applicant:
      IF applicant is eligible:
          APPEND ranking tuple to eligible_list

  SORT eligible_list by:
      aggregate descending,
      lowest_subject descending,
      attendance descending,
      income ascending,
      applicant_id ascending
```

```
  DISPLAY eligibility results
  DISPLAY first scholarship_count eligible applicants as winners
END
```

4. Complete Python Program

```
# University Scholarship Allocation
# Introductory Python only: variables, lists, loops, conditionals,
# operators, tuples, sorting, and tuple unpacking.

MIN_AGGREGATE = 75.0
MIN_SUBJECT_MARK = 60
MIN_ATTENDANCE = 85.0
MAX_FAMILY_INCOME = 300000


def read_number(prompt, minimum=None, maximum=None, whole_number=False):
    """Read a valid numeric value using a loop and conditionals."""
    while True:
        try:
            value = float(input(prompt))
            if whole_number:
                if value != int(value):
                    print("Enter a whole number.")
                    continue
                value = int(value)

            if minimum is not None and value < minimum:
                print("Value is below the allowed minimum.")
                continue
            if maximum is not None and value > maximum:
                print("Value is above the allowed maximum.")
                continue
            return value
        except ValueError:
            print("Invalid input. Please enter a number.")


def main():
    applicant_count = int(read_number(
        "Number of applicants: ", minimum=1, whole_number=True
    ))
    scholarship_count = int(read_number(
        "Number of scholarships available: ", minimum=0, whole_number=True
    ))

    # Each nested list stores one applicant's information.
    applicants = []

    for i in range(applicant_count):
        print("\\nApplicant", i + 1)
        applicant_id = input("Applicant ID: ").strip()
        name = input("Full name: ").strip()

        # Marks are out of 100.
        mark1 = read_number("Subject 1 marks (0-100): ", 0, 100)
        mark2 = read_number("Subject 2 marks (0-100): ", 0, 100)
        mark3 = read_number("Subject 3 marks (0-100): ", 0, 100)
        attendance = read_number("Attendance percentage (0-100): ", 0, 100)
        income = read_number("Annual family income in rupees: ", 0)

        aggregate = (mark1 + mark2 + mark3) / 3
        lowest_subject = min(mark1, mark2, mark3)

        # All conditions must be true for eligibility.
        eligible = (
            aggregate >= MIN_AGGREGATE
            and mark1 >= MIN_SUBJECT_MARK
            and mark2 >= MIN_SUBJECT_MARK
            and mark3 >= MIN_SUBJECT_MARK
            and attendance >= MIN_ATTENDANCE
            and income <= MAX_FAMILY_INCOME
        )

        if eligible:
            status = "Eligible"
        else:
            status = "Not eligible"

        # Store values in a nested list for later reporting.
```

```
        applicants.append([
            applicant_id, name, mark1, mark2, mark3,
            aggregate, lowest_subject, attendance, income,
            status
        ])

    # Ranking tuple:
    # (-aggregate, -lowest_subject, -attendance, income, ID, name)
    # Negative values make higher scores sort first in ascending order.
    ranking = []

    for applicant in applicants:
        (applicant_id, name, mark1, mark2, mark3, aggregate,
         lowest_subject, attendance, income, status) = applicant

        if status == "Eligible":
            ranking.append((
                -aggregate, -lowest_subject, -attendance,
                income, applicant_id, name
            ))

    ranking.sort()
    winners = ranking[:scholarship_count]

    print("\\nELIGIBILITY REPORT")
    print("-" * 72)
    print("ID       Name                 Aggregate  Attendance  Income    Status")
    print("-" * 72)

    for applicant in applicants:
        (applicant_id, name, mark1, mark2, mark3, aggregate,
         lowest_subject, attendance, income, status) = applicant
        print(
            applicant_id[:8].ljust(8),
            name[:20].ljust(20),
            f"{aggregate:8.2f}",
            f"{attendance:10.2f}",
            f"{income:9.0f}",
            status
        )

    print("\\nSCHOLARSHIP RANKING")
    print("-" * 72)

    if len(winners) == 0:
        print("No scholarships can be awarded based on the current inputs.")
    else:
        print("Rank  ID       Name                 Aggregate  Attendance  Income")
        for rank, winner in enumerate(winners, start=1):
            neg_aggregate, neg_lowest, neg_attendance, income, applicant_id, name = winner
            print(
                f"{rank:>4}  {applicant_id[:8].ljust(8)} "
                f"{name[:20].ljust(20)} "
                f"{-neg_aggregate:8.2f} "
                f"{-neg_attendance:10.2f} "
                f"{income:9.0f}"
            )

    eligible_total = len(ranking)
    awarded_total = min(scholarship_count, eligible_total)
    print("\\nEligible applicants:", eligible_total)
    print("Scholarships awarded:", awarded_total)


if __name__ == "__main__":
    main()
```

Input validation: the program rejects non-numeric entries and values outside the allowed mark, attendance, income, and count ranges. Applicant IDs should be unique for unambiguous reporting.

```
  v
END
```

7. Testing and Edge Cases

Test case	Expected behavior
Aggregate exactly 75%	Passes aggregate condition if all other criteria pass
Any subject mark is 59	Not eligible due to subject minimum
Attendance exactly 85%	Passes attendance condition
Income exactly ■300,000	Passes income condition
No eligible applicants	Prints that no scholarships can be awarded
More scholarships than eligible applicants	Awards only to the eligible applicants available
Tied aggregate	Compares lowest subject mark, then attendance, then income, then ID

8. Concepts Demonstrated

• Computational thinking: breaking the problem into input, validation, eligibility, ranking, and output tasks.
• Variables and data types: strings for IDs/names, integers or floats for marks and income, booleans for eligibility.
• Operators and conditionals: arithmetic, comparisons, and logical AND.
• Loops: repeated input, validation, reporting, and ranking output.
• Lists and nested lists: storing applicants and their fields.
• Tuples and unpacking: storing sortable ranking keys and unpacking applicant fields.
• Strings and pattern generation: formatted output and separator lines made with the * operator.

Conclusion

This solution demonstrates an end-to-end introductory Python approach to scholarship eligibility and ranking. It is transparent, uses explicit criteria, and applies a consistent tie-breaking order. The sample thresholds are placeholders and should be replaced with the university's official rules.

STUDENT TIMETABLE CONFLICT DETECTION

Case Study with Complete Python Solution

Course area: Introductory Python Programming

Topics: Algorithms, pseudocode, variables, data types, operators, conditionals, loops, strings, lists, nested lists, tuples, tuple operations, unpacking, and pattern generation.

Objective

Build a simple timetable validator that stores course schedules as tuples, detects overlapping classes for a student, checks room availability, and validates a proposed timetable. The solution uses only introductory Python constructs and does not require external libraries.

Schedule representation

Each class is stored as a tuple: (course_code, day, start_time, end_time, room). Days use names such as Monday. Times use 24-hour HH:MM strings, for example 09:00 and 10:30. A class occupies the interval from start time up to, but not including, end time. Therefore, a class ending at 10:00 and another starting at 10:00 do not overlap.

1. Problem Definition and Rules

Rule	Description
Valid time	Time must be in HH:MM format and within 00:00–23:59.
Valid interval	Start time must be earlier than end time.
Student conflict	Two courses conflict when they are on the same day and their time intervals overlap.
Room conflict	Two classes conflict for a room when they use the same room on the same day at overlapping times.
Room availability	A room is unavailable if a separate room-block tuple overlaps the proposed class.
Proposed timetable	Every class must have valid times, no student course conflict, no room conflict, and no room-block conflict.

2. Algorithm

```
1. Store each class as a tuple (course, day, start, end, room).
2. Convert HH:MM strings into minutes after midnight for comparison.
3. Validate each class's time format and ensure start < end.
4. Compare every pair of classes:
   a. If days match and time intervals overlap, report a student conflict.
   b. If rooms and days match and times overlap, report a room conflict.
5. Compare each class with room-block tuples to find unavailable rooms.
6. Validate the proposed timetable using all checks.
7. Print conflicts and a final VALID / INVALID result.
```

3. Pseudocode

```
START
  READ course schedules and room-block schedules
  FOR each class:
      CHECK time format and start time < end time

  FOR each pair of classes (i, j), where i < j:
      IF same day AND intervals overlap:
          REPORT student schedule conflict
      IF same room AND same day AND intervals overlap:
          REPORT room conflict

  FOR each class and each room block:
      IF same room AND same day AND intervals overlap:
          REPORT room unavailable

  IF no invalid times and no conflicts:
      PRINT "Proposed timetable is VALID"
  ELSE:
      PRINT "Proposed timetable is INVALID"
END
```

4. Complete Python Program

```
# Student Timetable Conflict Detection
# Introductory Python only: strings, lists, tuples, loops,
# conditionals, operators, functions, and tuple unpacking.

def time_to_minutes(time_text):
    """Convert valid HH:MM time text into minutes after midnight."""
    parts = time_text.split(":")
    if len(parts) != 2:
        return -1

    hour_text, minute_text = parts
    if not hour_text.isdigit() or not minute_text.isdigit():
        return -1

    hour = int(hour_text)
    minute = int(minute_text)

    if hour < 0 or hour > 23 or minute < 0 or minute > 59:
        return -1

    return hour * 60 + minute


def intervals_overlap(start1, end1, start2, end2):
    # Half-open intervals: touching endpoints are not a conflict.
    return start1 < end2 and start2 < end1


def valid_class(class_row):
    course, day, start_text, end_text, room = class_row
    start = time_to_minutes(start_text)
    end = time_to_minutes(end_text)

    if course == "" or day == "" or room == "":
        return False
    if start == -1 or end == -1:
        return False
    if start >= end:
        return False
    return True


def read_class(label):
    print("\\n" + label)
    course = input("Course code: ").strip()
    day = input("Day (e.g. Monday): ").strip().title()
    start = input("Start time (HH:MM, 24-hour): ").strip()
    end = input("End time (HH:MM, 24-hour): ").strip()
    room = input("Room: ").strip().upper()
    return (course, day, start, end, room)


def main():
    class_count = int(input("Number of classes in proposed timetable: "))
    while class_count < 1:
        print("Enter at least one class.")
        class_count = int(input("Number of classes: "))

    # A list of tuples: each tuple represents one course schedule.
    classes = []
    for i in range(class_count):
        classes.append(read_class("Class " + str(i + 1)))

    block_count = int(input("\\nNumber of room-unavailability periods: "))
    while block_count < 0:
        print("Count cannot be negative.")
        block_count = int(input("Number of room-unavailability periods: "))

    # A room block uses the same tuple structure.
    room_blocks = []
    for i in range(block_count):
        room_blocks.append(read_class("Room block " + str(i + 1)))

    issues = []

    # First validate all class and room-block records.
    for i, class_row in enumerate(classes):
        if not valid_class(class_row):
            issues.append(
                "Invalid class schedule at position " + str(i + 1)
            )

    for i, block in enumerate(room_blocks):
        if not valid_class(block):
```

```
            issues.append(
                "Invalid room-block schedule at position " + str(i + 1)
            )

    # Check course overlaps and room double-booking.
    for i in range(len(classes)):
        for j in range(i + 1, len(classes)):
            course1, day1, start1_text, end1_text, room1 = classes[i]
            course2, day2, start2_text, end2_text, room2 = classes[j]

            if not valid_class(classes[i]) or not valid_class(classes[j]):
                continue

            start1 = time_to_minutes(start1_text)
            end1 = time_to_minutes(end1_text)
            start2 = time_to_minutes(start2_text)
            end2 = time_to_minutes(end2_text)

            if day1 == day2 and intervals_overlap(
                start1, end1, start2, end2
            ):
                issues.append(
                    "STUDENT CONFLICT: " + course1 + " and " + course2
                    + " overlap on " + day1 + "."
                )

                if room1 == room2:
                    issues.append(
                        "ROOM CONFLICT: " + room1 + " is double-booked by "
                        + course1 + " and " + course2 + "."
                    )

    # Check whether a proposed class falls inside a room-block period.
    for class_row in classes:
        course, day, start_text, end_text, room = class_row
        if not valid_class(class_row):
            continue

        start = time_to_minutes(start_text)
        end = time_to_minutes(end_text)

        for block in room_blocks:
            block_name, block_day, block_start_text, block_end_text, block_room = block
            if not valid_class(block):
                continue

            block_start = time_to_minutes(block_start_text)
            block_end = time_to_minutes(block_end_text)

            if room == block_room and day == block_day:
                if intervals_overlap(start, end, block_start, block_end):
                    issues.append(
                        "ROOM UNAVAILABLE: " + room + " for " + course
                        + " on " + day + " (" + block_start_text
                        + "-" + block_end_text + " blocked)."
                    )

    print("\\nPROPOSED TIMETABLE")
    print("=" * 76)
    print("Course".ljust(12), "Day".ljust(12), "Start".ljust(8),
          "End".ljust(8), "Room".ljust(10), "Status")
    print("-" * 76)

    for class_row in classes:
        course, day, start_text, end_text, room = class_row
        if valid_class(class_row):
            status = "Time valid"
        else:
            status = "Invalid time"
        print(course.ljust(12), day.ljust(12), start_text.ljust(8),
              end_text.ljust(8), room.ljust(10), status)

    print("\\nVALIDATION RESULTS")
    print("-" * 76)
    if len(issues) == 0:
        print("No conflicts found.")
        print("PROPOSED TIMETABLE: VALID")
    else:
        # Display issue list; duplicates can be useful for showing
        # both student and room conflicts for the same pair.
        for issue in issues:
            print("-", issue)
        print("PROPOSED TIMETABLE: INVALID")

    print("\\nTotal issues found:", len(issues))

if __name__ == "__main__":
```

```
main()
```

Input assumption: times must use 24-hour HH:MM format, such as 08:30. The program treats the schedule as one student's proposed course list; a conflict between any two courses therefore makes the student's timetable invalid.

Test case	Expected behavior
09:00–10:00 and 10:00–11:00	No overlap; endpoints just touch.
09:00–10:00 and 09:45–10:15, same day	Student conflict.
Same time but different days	No student conflict.
Same room, same day, overlapping times	Student conflict and room double-booking.
Class overlaps a room block	Room-unavailability issue.
Start time equals end time	Invalid schedule.
Time written as 9 AM	Invalid format; use 09:00.

8. Syllabus Concepts Demonstrated

• Computational thinking: separating input, validation, overlap detection, room checks, and reporting.
• Variables and data types: strings for course/day/time/room; integers for converted minutes; booleans for validation decisions.
• Operators and conditionals: comparisons and logical AND/OR-style decision checks.
• Loops: iterating over classes, pairs of classes, and room blocks.
• Strings: splitting times and formatting output.
• Lists and nested lists: storing schedules and issue messages.
• Tuples, tuple operations, and unpacking: representing each schedule and assigning tuple fields to variables.
• Pattern generation: report separator lines created with the string repetition operator.

Conclusion

This program checks whether a proposed student timetable has overlapping courses, room double-bookings, or room-unavailability conflicts. Its interval rule is explicit and its validation output explains why a timetable is invalid.

MANUFACTURING COST AND PROFITABILITY ANALYSIS

Case Study with Complete Python Solution

Course area: Introductory Python Programming

Topics used: Algorithms, pseudocode, variables, data types, operators, conditionals, loops, strings, lists, nested lists, tuples, tuple operations, unpacking, and pattern generation.

Objective

Develop a program that calculates manufacturing costs for a batch of products. It includes material costs, labour charges, wastage, machine costs, selling price, revenue, profit, profit margin, and whether the production batch meets a target profit percentage.

Assumptions

All rates and quantities are entered by the user. Wastage is calculated as a percentage of the base material cost. Labour and machine costs are entered as batch totals. The target is defined as a profit margin percentage of revenue. These assumptions can be changed to match a factory's accounting policy.

1. Problem Definition

For each product type, the program reads the product name, quantity produced, material cost per unit, wastage percentage, labour cost per unit, machine cost for the batch, and selling price per unit. It calculates total cost and profit, then checks whether the profit margin meets the target.

Formulae

Measure	Formula
Base material cost	Quantity × material cost per unit
Wastage cost	Base material cost × wastage percentage ÷ 100
Total labour cost	Quantity × labour cost per unit
Total manufacturing cost	Material + wastage + labour + machine cost
Total sales revenue	Quantity × selling price per unit
Profit / loss	Sales revenue – total manufacturing cost
Profit margin (%)	Profit ÷ sales revenue × 100 (when revenue > 0)

2. Algorithm

```
1. Read the number of product batches and target profit margin.
2. For each batch, read product details and cost inputs.
3. Calculate base material cost and wastage cost.
4. Calculate labour cost, machine cost, total cost, and revenue.
5. Calculate profit/loss and profit margin.
6. Compare profit margin with the target.
7. Store the results in a nested list.
8. Display a cost report for every product and a summary.
```

3. Pseudocode

```
START
  INPUT number_of_products, target_margin
  results = empty list

  FOR each product:
      INPUT name, quantity, material_rate, wastage_percent
      INPUT labour_rate, machine_cost, selling_price
      material_cost = quantity * material_rate
      wastage_cost = material_cost * wastage_percent / 100
      labour_cost = quantity * labour_rate
      total_cost = material_cost + wastage_cost
                   + labour_cost + machine_cost
      revenue = quantity * selling_price
      profit = revenue - total_cost

      IF revenue > 0:
          margin = profit / revenue * 100
      ELSE:
          margin = 0

      IF margin >= target_margin AND profit >= 0:
          status = "Target met"
      ELSE:
          status = "Target not met"

      APPEND all results to results

  DISPLAY each product's costs, revenue, profit, margin, status
END
```

4. Complete Python Program

```
# Manufacturing Cost and Profitability Analysis
# Uses introductory Python constructs only.

def read_number(prompt, minimum=0):
    """Keep asking until a valid number at or above minimum is entered."""
    while True:
        try:
            value = float(input(prompt))
            if value < minimum:
                print("Enter a value greater than or equal to", minimum)
            else:
                return value
        except ValueError:
            print("Invalid input. Please enter a number.")


def main():
    product_count = int(read_number(
        "Number of product batches: ", 1
    ))
    target_margin = read_number(
        "Target profit margin percentage (e.g. 15 for 15%): ", 0
    )

    # Each row in this nested list stores one product's results.
    results = []

    for i in range(int(product_count)):
        print("\\nPRODUCT BATCH", i + 1)
        name = input("Product name: ").strip()
        if name == "":
            name = "Product " + str(i + 1)

        quantity = int(read_number("Quantity produced (units): ", 1))
        material_rate = read_number("Material cost per unit: ", 0)
        wastage_percent = read_number("Wastage percentage (0-100): ", 0)
        while wastage_percent > 100:
            print("Wastage percentage cannot exceed 100.")
            wastage_percent = read_number(
                "Wastage percentage (0-100): ", 0
            )

        labour_rate = read_number("Labour cost per unit: ", 0)
        machine_cost = read_number("Machine cost for this batch: ", 0)
        selling_price = read_number("Selling price per unit: ", 0)

        material_cost = quantity * material_rate
        wastage_cost = material_cost * wastage_percent / 100
        labour_cost = quantity * labour_rate
        total_cost = (
            material_cost + wastage_cost + labour_cost + machine_cost
        )
        revenue = quantity * selling_price
        profit = revenue - total_cost

        if revenue > 0:
            margin = (profit / revenue) * 100
        else:
            margin = 0.0

        if profit >= 0 and margin >= target_margin:
            status = "Target met"
        else:
            status = "Target not met"

        # Tuple demonstrates grouping and tuple unpacking.
        result = (
            name, quantity, material_cost, wastage_cost, labour_cost,
            machine_cost, total_cost, revenue, profit, margin, status
        )
        results.append(result)

    print("\\nMANUFACTURING COST REPORT")
    print("=" * 104)
    print(
        "Product".ljust(18), "Qty".rjust(6),
        "Material".rjust(10), "Wastage".rjust(10),
        "Labour".rjust(10), "Machine".rjust(10),
        "Total Cost".rjust(12), "Revenue".rjust(12),
        "Profit".rjust(10), "Margin%".rjust(9), "Status"
    )
    print("-" * 104)

    total_cost_all = 0
```

```
    total_revenue_all = 0
    total_profit_all = 0

    for result in results:
        (name, quantity, material_cost, wastage_cost, labour_cost,
         machine_cost, total_cost, revenue, profit, margin, status) = result

        print(
            name[:18].ljust(18),
            str(quantity).rjust(6),
            f"{material_cost:10.2f}",
            f"{wastage_cost:10.2f}",
            f"{labour_cost:10.2f}",
            f"{machine_cost:10.2f}",
            f"{total_cost:12.2f}",
            f"{revenue:12.2f}",
            f"{profit:10.2f}",
            f"{margin:9.2f}",
            status
        )

        total_cost_all += total_cost
        total_revenue_all += revenue
        total_profit_all += profit

    if total_revenue_all > 0:
        overall_margin = total_profit_all / total_revenue_all * 100
    else:
        overall_margin = 0.0

    print("=" * 104)
    print("TOTAL COST:", f"{total_cost_all:.2f}")
    print("TOTAL REVENUE:", f"{total_revenue_all:.2f}")
    print("TOTAL PROFIT / LOSS:", f"{total_profit_all:.2f}")
    print("OVERALL PROFIT MARGIN:", f"{overall_margin:.2f}%")

    if total_profit_all >= 0 and overall_margin >= target_margin:
        print("OVERALL RESULT: Target profit margin achieved.")
    else:
        print("OVERALL RESULT: Target profit margin not achieved.")


if __name__ == "__main__":
    main()
```

Note: the program assumes quantity is a whole number. The wastage percentage is restricted to 0–100. For production accounting, confirm whether machine cost includes setup, maintenance, depreciation, or energy before using real data.

7. Testing and Edge Cases

Test	Expected behavior
Wastage is 0%	Wastage cost is zero
Selling price is 0	Revenue is zero; margin is set to 0 to avoid division by zero
Revenue is below total cost	Profit is negative and target is not met
Margin exactly equals target	Target is met if profit is non-negative
Wastage percentage exceeds 100	Program asks for a valid percentage
Invalid text in numeric field	Program asks again
Several product batches	Each is reported and totals are accumulated

8. Syllabus Concepts Demonstrated

• Computational thinking: splitting the task into input, cost calculation, profitability decision, and reporting.
• Variables and data types: product names as strings, quantities as integers, cost values as floats.
• Operators: arithmetic and comparison operators.
• Conditional statements: selecting whether each product meets the target.
• Loops: repeated input, validation, and report generation.
• Strings and pattern generation: labels, formatting, and separator lines using the * operator.
• Lists and nested lists: storing multiple product results.
• Tuples and unpacking: grouping each result and assigning tuple values to variables.

Conclusion

The program calculates batch-level manufacturing cost, sales revenue, profit or loss, and profit margin, then evaluates each product and the overall production result against a target. The formulas and assumptions should be checked against the organization's costing policy before practical use.

Case Study 65

Campus Resource Allocation Under Constraints

Complete Python Solution Using Only the Assigned Syllabus

1. Problem Definition

A campus has a limited stock of shared equipment, such as projectors, laptops, and microphones. Several activities request different quantities of each resource. The program must process activities in a specified priority order, allocate what is available, calculate shortages, and show whether each activity can be fully supported. A feasible allocation never assigns more units than the campus owns. If resources are insufficient, the report identifies the unmet quantities.

2. Resource Rules and Example Data

Resources are tracked separately by type. Activities are processed from highest to lowest priority; a smaller priority number means higher priority. Within each activity, resources are considered in the order Projectors, Laptops, Microphones. The example uses whole-number equipment quantities.

Resource	Available stock
Projectors	5
Laptops	8
Microphones	4

Activity	Priority	Projectors	Laptops	Microphones
Science Fair	1	2	3	2
Coding Workshop	2	1	5	1
Cultural Event	3	3	2	3
Club Meeting	4	1	2	1

Important: this is a classroom example. The allocation is greedy and priority-based, not a general optimization algorithm. A lower-priority activity may receive a partial allocation when earlier activities use the available stock.

3. Computational Thinking

Decomposition: store stock, process each activity, allocate resource types, calculate shortages, and summarize.
Pattern recognition: the same allocation steps apply to each activity and resource type.
Abstraction: represent each activity as a tuple and resource quantities as nested lists.
Algorithm design: sort by the given priority order, allocate no more than the remaining stock, and record requested, allocated, and shortage quantities.

4. Algorithm

1. Start and define resource names and available quantities.
2. Store each activity as a tuple containing its name, priority, and requested quantities.
3. Sort activities into ascending priority order using simple loops and tuple comparisons.
4. For each activity and each resource, allocate the smaller of requested quantity and remaining stock.
5. Subtract the allocated amount from the remaining stock and calculate shortage = requested – allocated.
6. Store the allocation and shortage values in nested lists.
7. Mark an activity Fully Allocated if all shortages are zero; otherwise mark it Partial.
8. Display each activity's request, allocation, and shortage, then show remaining stock.
9. Stop.

5. Pseudocode

```
START
  resource_names = [Projectors, Laptops, Microphones]
  available = [5, 8, 4]
  activities = list of (name, priority, request_list) tuples
```

```
  arrange activities by ascending priority
  allocations = empty list
  shortages = empty list

  FOR each activity in priority order
      UNPACK name, priority, requests
      allocated_row = empty list
      shortage_row = empty list

      FOR each resource index j
          allocated = minimum(requests[j], available[j])
          shortage = requests[j] - allocated
          available[j] = available[j] - allocated
          APPEND allocated to allocated_row
          APPEND shortage to shortage_row
      END FOR

      APPEND rows to allocations and shortages
  END FOR

  DISPLAY requests, allocations, shortages, remaining stock
END
```

```
            temp = ordered[j]
            ordered[j] = ordered[j + 1]
            ordered[j + 1] = temp

remaining = initial_stock[:]
allocation_results = []
shortage_results = []

for activity in ordered:
    name, priority, requests = activity  # tuple unpacking
    allocated_row = []
    shortage_row = []

    for j in range(len(resource_names)):
        requested = requests[j]

        # Allocate only what remains in stock.
        if requested <= remaining[j]:
            allocated = requested
        else:
            allocated = remaining[j]

        shortage = requested - allocated
        remaining[j] = remaining[j] - allocated

        allocated_row.append(allocated)
        shortage_row.append(shortage)

    allocation_results.append(allocated_row)
    shortage_results.append(shortage_row)

print("CAMPUS RESOURCE ALLOCATION REPORT")
print("=" * 90)
print("Priority order: smaller number means higher priority")
print("-" * 90)

for i in range(len(ordered)):
    name, priority, requests = ordered[i]
    allocated = allocation_results[i]
    shortage = shortage_results[i]

    if shortage[0] == 0 and shortage[1] == 0 and shortage[2] == 0:
        status = "Fully Allocated"
    else:
        status = "Partial"

    print("\nActivity:", name, "| Priority:", priority)
    print("Resource       Requested   Allocated   Shortage")
    print("-" * 48)

    for j in range(len(resource_names)):
        print("{:<14} {:>9} {:>11} {:>10}".format(
            resource_names[j], requests[j], allocated[j], shortage[j]))

    print("Status:", status)

print("\n" + "=" * 90)
print("REMAINING STOCK")
for j in range(len(resource_names)):
    print(resource_names[j] + ":", remaining[j])

print("\nTOTAL SHORTAGES BY RESOURCE")
total_shortages = [0, 0, 0]
for row in shortage_results:
    for j in range(len(resource_names)):
        total_shortages[j] = total_shortages[j] + row[j]

for j in range(len(resource_names)):
    print(resource_names[j] + ":", total_shortages[j])

print("\nACTIVITIES WITH SHORTAGES")
found = False
for i in range(len(ordered)):
    if shortage_results[i][0] > 0 or shortage_results[i][1] > 0 or shortage_results[i][2] > 0:
        print(ordered[i][0])
        found = True
if not found:
    print("None; all requests were met.")
```

8. Sample Allocation Results

Activities are allocated in priority order: Science Fair, Coding Workshop, Cultural Event, then Club Meeting. The program always respects available stock; it never allocates a negative quantity or more than remains.

Activity	Priority	Resource	Requested	Allocated	Shortage
Science Fair	1	Projectors	2	2	0
Science Fair	1	Laptops	3	3	0

Activity	Priority	Resource	Requested	Allocated	Shortage
Science Fair	1	Microphones	2	2	0
Coding Workshop	2	Projectors	1	1	0
Coding Workshop	2	Laptops	5	5	0
Coding Workshop	2	Microphones	1	1	0
Cultural Event	3	Projectors	3	2	1
Cultural Event	3	Laptops	2	0	2
Cultural Event	3	Microphones	3	1	2
Club Meeting	4	Projectors	1	0	1
Club Meeting	4	Laptops	2	0	2
Club Meeting	4	Microphones	1	0	1

Final stock: Projectors = 0, Laptops = 0, Microphones = 0.
Total shortages: Projectors = 2, Laptops = 4, Microphones = 3.
Science Fair and Coding Workshop are fully allocated. Cultural Event and Club Meeting receive partial allocations.

9. Test Cases

Test	Situation	Expected behavior
1	Request is less than stock	Allocate full request; shortage is 0
2	Request equals remaining stock	Allocate all remaining stock
3	Request exceeds stock	Allocate remaining stock only; report difference as shortage
4	Resource stock is 0	Allocation is 0; all requested units are shortage
5	Multiple activities compete	Higher-priority activity is processed first
6	All requests fit stock	All activities are fully allocated; remaining stock may be positive

10. Syllabus Concepts Used

• Computational thinking: decompose stock tracking, allocation, shortage calculation, and reporting.

• Algorithms, pseudocode, and flowcharts: plan and represent the allocation procedure.

• Variables and data types: activity names (strings), priorities and quantities (integers), and lists.

• Operators: arithmetic subtraction and comparisons.

• Conditional statements: choose allocated quantity and determine full or partial allocation.

• Loops: process activities, resources, and result rows.

• Strings and pattern generation: report labels and repeated separator lines.

• Lists and nested lists: stock vectors, resource requests, allocation rows, and shortage rows.

• Tuples and tuple operations: fixed activity records; access priority by index.

• Unpacking: extract name, priority, and requests from each activity tuple.

11. Conclusion

The program produces a priority-based feasible allocation, computes shortages for every activity and resource, and reports remaining stock. Its key constraint is that the amount allocated to a resource never exceeds the amount available when that activity is processed. It uses only introductory Python constructs from the assigned syllabus.